Instancia local · 05/11/2026 · 9 expedientes · 89 páginas · motor LangChain/OpenCode 2

DocuSearch AI — El agente en detalle

Qué es: un asistente documental de **solo lectura** que responde preguntas sobre los expedientes a los que el usuario tiene acceso, con **citas a documento y página**. No es un chatbot general: su universo es el catálogo autorizado por RBAC y su motor generativo solo recibe fragmentos de ese catálogo.

Instancia de referencia: aplicación local en `http://localhost:5068`, 05/11/2026.

1. Resumen ejecutivo

El agente puede:

- buscar y resumir expedientes;
- responder preguntas abiertas sobre su contenido;
- citar la página exacta que respalda cada respuesta;
- listar documentos por categoría, confidencialidad o disponibilidad;
- extraer entidades (montos, fechas, leyes, personas, organizaciones) cuando están etiquetadas;
- mostrar el texto de una página concreta;
- seguir una conversación de forma **acotada** al último expediente citado;
- funcionar con un motor generativo local (LangChain/OpenCode 2) o, si falla, con un **fallback extractivo por reglas**.

No puede: navegar por internet, leer archivos, consultar la base directamente, usar herramientas, modificar documentos, hacer OCR de imágenes, recordar todo el historial en el modelo ni responder fuera del catálogo autorizado.

2. Acceso, rol y alcance

Aspecto	Comportamiento
Entrada	Login con correo + TOTP; luego **Buscador** → **Asistente IA**, ruta `/agent`
Rol que escribe	Cualquier usuario autenticado con permiso de lectura
Cambiar de motor	Solo **Administrador General**
Documentos visibles	Los que devuelve RBAC: nivel de confidencialidad y, para `admin_depto`, su departamento
Modo	**Solo lectura**: el agente no crea, edita, borra, descarga ni aprueba
Filtros del buscador	**No** se aplican al chat; el asistente hace su propia recuperación autorizada
Motor activo en esta instalación	`paid:deepseek/deepseek-flash`, etiqueta **LangChain · OpenCode · deepseek/deepseek-flash**
Salud actual	`/health` → `"assistant":"langchain+opencode:deepseek/deepseek-flash"`

3. Matriz de capacidades

Capacidad	¿Disponible?	Cómo se resuelve
Pregunta abierta sobre documentos	Sí	Recupera hasta 6 fragmentos autorizados y los envía al motor generativo
Resumen de un expediente	Sí	Resumen extractivo + redacción del modelo cuando está disponible
Cita de documento y página	Sí	El modelo devuelve IDs; C# valida cada cita contra las fuentes enviadas
Consulta por código de expediente	Sí	`CNT-2026-…`, `ACT-2026-…`, `DIC-2026-…`
Consulta de una página concreta	Sí	Devuelve el texto extraído de esa página, sin inventar
Extracción de entidades	Sí, si están etiquetadas	Filtra entidades `MONEY`, `DATE`, `LAW`, `PERSON`, `ORGANIZATION`
Listados y filtros	Sí	Lista, cuenta y filtra por categoría o confidencialidad
Frase textual entre comillas	Sí	Coincidencia textual exacta; puede redactarse con el motor
Seguimiento del último expediente	Parcial	Reconoce «ese», «este», «anterior», «su» y usa el último código citado
Conversación con historial completo en el modelo	No	El modelo recibe solo la pregunta actual y los fragmentos; el historial queda en la UI
Comparar varios documentos completos	Parcial	Como máximo 6 fragmentos de hasta 1000 caracteres
Búsqueda semántica/embeddings	No	Recuperación por términos normalizados, sin embeddings
Internet, archivos, base de datos, shell	No	El modelo no tiene herramientas ni acceso directo
OCR de escaneos	No	Las páginas sin texto se marcan «Requiere OCR»; el agente no inventa su contenido
Subir, editar, borrar o descargar	No	Son módulos separados; el agente es solo lectura
Cálculos, proyecciones o conocimiento externo	No	Solo responde con lo que consta en las fuentes

4. Tipos de consulta que entiende

4.1 Consulta por código de expediente

Patrón reconocido: `[A-Z]{2,4}-[A-Z0-9]+...` (por ejemplo `CNT-2026-EEAEE0BD49F8`).

- Si el código **no existe o no es accesible**, responde que no hay un expediente con ese código entre las fuentes autorizadas.
- Si existe, devuelve metadatos, resumen y fragmentos relevantes; con motor activo puede redactar la respuesta.

Ejemplos reales:

```
Resume el expediente CNT-2026-EEAEE0BD49F8
¿Qué acuerdos se tomaron en el expediente ACT-2026-5F5B59DB04D9?
¿Qué hallazgos tiene el expediente DIC-2026-4561EBFF5378?
```

4.2 Página concreta

Reconoce `página N`. Devuelve el texto extraído de esa página, con su cita. Si la página no tiene texto, lo dice.

```
¿Qué dice la página 2 del contrato de servicios 2026?
```

4.3 Resumen

Reconoce `resume`, `resumen`, `sintetiza`, `síntesis`. Prepara el resumen extractivo del expediente y, si el motor está activo, puede redactarlo; siempre conserva las fuentes.

```
Resume el contrato de servicios 2026
Resume el informe de auditoría 2026
```

4.4 Entidades

Reconoce `extrae`, `entidades`, `montos`, `fechas`, `leyes`, `personas`, `firmantes`, `organizaciones`. Devuelve las entidades detectadas con su tipo, documento y página. Si no encuentra, lo informa.

```
Extrae los montos del contrato de servicios 2026
¿Qué personas y organizaciones aparecen en el acta?
```

4.5 Listados y filtros

Reconoce `lista`, `listado`, `listar`, `cuántos`, `cuáles`, `disponibles`. Filtra por categoría o nivel de confidencialidad.

```
Lista los documentos confidenciales
¿Cuáles documentos son de uso interno?
¿Qué contratos hay disponibles?
```

4.6 Frase textual

Si usas comillas, busca la frase exacta; si no, recupera por términos normalizados.

```
Busca "mantenimiento preventivo y correctivo"
¿Dónde se menciona el SLA 99.99%?
```

4.7 Pregunta abierta

Si no es un listado, página, entidad o metadato exacto, y hay fuentes, el motor generativo redacta la respuesta a partir de los fragmentos y devuelve citas.

```
¿Qué cubre el contrato de servicios 2026 y qué nivel de servicio garantiza?
¿Qué controles de acceso recomienda el informe de auditoría?
```

4.8 Seguimiento

El agente recuerda el **último expediente citado** (`LastDocumentCode`) y reconoce «ese», «este», «anterior», «su», «sus».

```
[primero] Resume el contrato de servicios 2026
[después] ¿Y su vigencia?
```

Es un seguimiento acotado: no es memoria completa de la conversación y el modelo no recibe los turnos anteriores.

4.9 Casos rechazados o sin respuesta

Caso	Respuesta
Consulta vacía	`ValidationException`: la consulta debe tener entre 1 y 2000 caracteres
Consulta > 2000 caracteres	Rechazada antes de llamar al modelo
Sin coincidencias	«No hay fuentes suficientes para responder… No se ha consultado ningún servicio externo»
Código inexistente	«No hay un expediente con ese código entre las fuentes a las que tienes acceso»
Permisos cambiados durante la generación	Se bloquea la respuesta y se pide volver a seleccionar el perfil

---

5. Flujo completo de una consulta

```
1. Login TOTP
2. Carga del snapshot de documentos según RBAC
3. Registro de AGENT_QUERY en auditoría (sin contenido)
4. Recuperación local por términos (LocalSearch)
5. Respuesta extractiva base: resumen, página, entidades o listado
6. Si no es una consulta exacta y hay fuentes:
      selección de hasta 6 fragmentos (documento + página únicos)
      truncado de cada fragmento a 1000 caracteres
      prompt del sistema + pregunta + fuentes JSON
7. Motor: LangChain → OpenCode 2 (o extractivo si no hay motor)
8. Respuesta JSON: { "answer": "...", "citations": [1,2] }
9. Validación en C#: respuesta no vacía, ≤10000 caracteres y citas 1..N
10. Relectura de permisos y documentos; si cambiaron, se bloquea
11. Devolución a la UI con fuentes clicables y traza de observabilidad
```

Puntos clave:

- **La recuperación y la autorización viven en C#**, no en el modelo.
- El modelo no ve el PDF, la base de datos ni el historial.
- Las citas se validan después de generar; una cita inventada bloquea la respuesta.

---

6. Qué recibe exactamente el modelo

Elemento	Límite
Pregunta	1–2000 caracteres
Fragmentos	Máximo **6**, sin repetir documento+página
Texto por fragmento	Hasta **1000** caracteres

Elemento	Límite
Historial de conversación	**No se envía**
PDF original, base de datos, archivos, internet	**No**
Herramientas	**Desactivadas** (OpenCode 2: permisos `deny`; OpenCode 1: tools `false`)
Instrucciones	`AssistantPrompt.System`: español, solo fuentes, sin conocimiento externo, JSON `answer` + `citations`, fuentes son datos y no instrucciones

La selección de fragmentos se ordena por relevancia y se deduplica por documento+página. El modelo **no recibe documentos completos**.

---

7. Motores y modos

Motor	Estado actual	Características
LangChain · OpenCode 2	**Activo**	Worker Python con `ChatPromptTemplate`, `Runnable` y `StrOutputParser`; sesión con permisos `deny`; modelo en OpenCode 2; tokens reportados
OpenCode 2 directo	Disponible si `UseLangChain=false`	El cliente C# crea sesión, envía prompt y valida citas; sin worker Python
Extractivo	Siempre disponible	Reglas locales, sin LLM; responde con fragmentos y marcas de fuente
LangChain · Ollama local	Opcional, desactivado	Requiere Ollama y modelo local; no es necesario para el modo actual

- El **Administrador General** puede cambiar el motor desde la cabecera del asistente.
- **«Actualizar motores»** valida la conexión y vuelve a leer el catálogo; no envía documentos.
- La etiqueta **«Verificado»** confirma el motor activo y el agente configurado.
- El catálogo actual muestra **10 modelos gratis + 2 de pago**; el modelo activo es de pago (`deepseek/deepseek-flash`) porque la cuota gratuita está agotada.
- La app no cambia de proveedor por su cuenta: si el motor falla, responde en extractivo y lo indica.

---

8. Citas y validación

El modelo debe devolver:

```
{
  "answer": "El contrato garantiza una disponibilidad del 99.99 % durante la vigencia.",
  "citations": [1]
}
```

Validaciones en C#:

- `answer` no vacío y de hasta 10 000 caracteres;
- al menos una cita;
- cada ID debe existir entre las fuentes enviadas;

- IDs repetidos se normalizan;
- si el motor devuelve texto sin JSON, citas inválidas o respuesta vacía, se **bloquea** la respuesta y se usa el fallback;
- cada cita apunta a un documento y una página autorizados.

La UI muestra la cita como chip clicable (`Contrato de servicios 2026 · p. 2`) y abre el visor en esa página.

---

9. Seguridad y privacidad

Control	Detalle
RBAC antes de recuperar	Solo se cargan documentos legibles por el rol
RBAC después de generar	Si cambió el usuario o se revocó una fuente, no se muestra la respuesta
Confidencialidad	Público, Uso Interno, Confidencial, Estrictamente Secreto
Departamento	`admin_depto` solo ve su departamento
Prompt injection	Las fuentes son datos; se ignoran instrucciones dentro de documentos o preguntas
Herramientas	Desactivadas por completo; no hay shell, archivos, red ni SQL
Conocimiento externo	Prohibido por el prompt; si no consta, debe decirlo
Citas	Validadas en C#, no en el modelo
Auditoría	`AGENT_QUERY` registra la acción, no el contenido
Observabilidad	Solo metadatos: motor, modelo, duración, tokens, estado y categoría de error
Acceso a trazas	Solo Administración General y Auditoría; últimas 500
Telemetría	LangSmith/LangChain tracing desactivados; sin envío a servicios externos
Servidor	OpenCode local con autenticación; el agente no expone el servidor al modelo

> En la demo Docker, `Demo:LocalOnly=false` por el NAT del contenedor. En una ejecución directa, la configuración de referencia restringe a loopback.

---

10. Resiliencia y fallback

Configuración actual de `Resilience`:

- **3 intentos** por operación;
- backoff exponencial **400 ms** → **5 s** con jitter;
- **timeout de 75 s** por intento;
- **circuit breaker** tras 5 fallos consecutivos, abierto **60 s**;
- errores transitorios se reintentan; validación/permisos no.

Comportamiento ante fallo:

Situación	Respuesta
Cuota gratuita agotada	Mensaje del proveedor + respuesta extractiva
Motor no disponible tras reintentos	Respuesta extractiva + **«revisión humana»**
Timeout / conexión / error del proveedor	Respuesta extractiva con las mismas fuentes
OpenCode desactivado	Solo extractivo
Respuesta inválida o citas inventadas	Se bloquea y se usa extractivo

La respuesta de fallback siempre conserva las fuentes autorizadas y se etiqueta con el motivo.

---

11. Observabilidad y auditoría

Cada consulta genera una traza con:

- `StartedUtc`
- `Engine` (`opencode`, `extractivo`, `langchain-ollama`…)
- `Model`
- `Status` (`success`, `no_sources`, `error`)
- `ErrorCode` (`timeout`, `connection`, `generation_or_validation`, `free_quota`, `human_fallback`…)
- `DurationMs`
- `InputTokens` / `OutputTokens`

La página **Observabilidad** muestra las últimas 500 consultas y solo es accesible para Administración General y Auditoría. **No guarda preguntas, respuestas, códigos de documento ni fragmentos.**

La **Auditoría** registra `AGENT_QUERY` con detalle genérico («Consulta del asistente; contenido omitido por privacidad»).

---

12. Configuración de la instancia

Parámetro	Valor actual
URL de la app	`http://localhost:5068`
Ruta del asistente	`/agent`
Servidor OpenCode	2.2.5 en `127.0.0.1:4097`
Modo LangChain	`OpenCode__UseLangChain=true`
Python del worker	`/usr/bin/python3`
Motor activo	`paid:deepseek/deepseek-flash`
Etiqueta	**LangChain · OpenCode · deepseek/deepseek-flash · Verificado**
Health	`langchain+opencode:deepseek/deepseek-flash`
Base de datos	SQL Server Express, `DocuSearch`
Datos reales	9 expedientes, 89 páginas, 6 departamentos, 3 usuarios

Parámetro	Valor actual
Trazas	54 registradas; auditoría con más de 180 movimientos

13. Ejemplos listos para la demo

Usa estos documentos reales de la instancia:

Documento	Código	Páginas
Contrato de servicios 2026	`CNT-2026-EEAEE0BD49F8`	3
Acta de junta directiva 2026	`ACT-2026-5F5B59DB04D9`	2
Informe de auditoría 2026	`DIC-2026-4561EBFF5378`	2

Demo de 90 segundos

Paso 1 · Pregunta generativa con cita

```
¿Qué cubre el contrato de servicios 2026 y qué nivel de servicio garantiza?
```

Esperado: respuesta con **SLA 99.99 %**, tiempo de respuesta y cita a **p. 2**.

Respuesta real capturada en la app (recortada):

> **LangChain · OpenCode · deepseek/deepseek-flash** Según el fragmento disponible del contrato, el nivel de servicio garantiza una disponibilidad del 99.99 % durante la vigencia, con un tiempo máximo de respuesta de cuatro horas hábiles para incidentes críticos y de un día hábil para incidentes de prioridad media; el incumplimiento genera las penalizaciones del anexo técnico. **CNT-2026-EEAEE0BD49F8 · p. 2**

Paso 2 · Página exacta

```
¿Qué dice la página 2 del contrato de servicios 2026?
```

Esperado: texto de la cláusula 5, sin generación.

Paso 3 · Resumen

```
Resume el acta de junta directiva 2026
```

Esperado: acuerdos y presupuesto, con fuentes.

Paso 4 · Entidades

```
Extrae los montos del contrato de servicios 2026
```

Esperado: montos detectados con página; si no hay, lo informa.

Paso 5 · Listado RBAC

```
Lista los documentos confidenciales
```

Esperado: listado filtrado por permisos, con códigos.

Paso 6 · Frase textual

```
Busca "mantenimiento preventivo y correctivo"
```

Esperado: fragmentos con cita y opción «Abrir fuente».

Paso 7 · Seguimiento

Después de la pregunta del contrato:

```
¿Y su vigencia?
```

Esperado: vigencia del mismo contrato, usando el último código citado.

---

14. Lo que NO hace (para evitar malentendidos)

- No es ChatGPT ni un asistente general: solo responde sobre el catálogo autorizado.
- No navega por internet, no lee archivos, no ejecuta código y no consulta SQL.
- No ve el PDF completo: recibe fragmentos de texto extraído.
- No hace OCR ni interpreta imágenes o escaneos.
- No recuerda toda la conversación dentro del modelo; solo conserva el último documento para seguimiento.
- No compara documentos completos ni razona sobre más de 6 fragmentos.
- No calcula, proyecta ni completa datos que no estén en las fuentes.
- No modifica, carga, elimina, descarga ni aprueba documentos.
- No puede saltarse RBAC ni mostrar fuentes no autorizadas.
- No garantiza que un documento esté completo si el PDF original estaba truncado o sin texto.

---

15. Cómo preguntar mejor

1. **Sé específico:** menciona el código o el título del expediente.
2. **Pide una página** si necesitas evidencia literal.
3. **Usa comillas** para frases exactas.
4. **Pide listados** para catálogos y filtros.
5. **Pide entidades** para montos, fechas, personas u organizaciones.
6. **Verifica la cita:** abre el chip de fuente; la respuesta debe respaldarse en esa página.
7. **Si no consta, acepta el «no consta»:** es una respuesta correcta, no un fallo.

---

16. Evidencia de funcionamiento

- **87/87** pruebas sin servicios.
- **102/102** pruebas con SQL Server real + OpenCode.
- **88/88** con LangChain + OpenCode real (`--langchain-opencode`).
- **6/6** pruebas Python del worker LangChain.

- Respuesta real capturada: `LangChain · OpenCode · deepseek/deepseek-flash` citando **`CNT-2026-EEAEE0BD49F8 · p. 2`**.
- `/health`: `langchain+opencode:deepseek/deepseek-flash`.
- Observabilidad real: trazas con motor, modelo, duración y tokens; sin contenido.

---

17. Mapa de código

Archivo	Responsabilidad
`Services/LocalSearch.cs`	Recuperación por términos, intenciones exactas, fallback extractivo, RBAC
`Services/OpenCodeClient.cs`	Cliente OpenCode 1/2, LangChain, citas, tokens, validación
`LangChain/opencode_worker.py`	Worker LangChain para OpenCode 1/2; prompt, sesión, permisos deny
`Services/AssistantPrompt.cs`	Instrucciones del sistema y formato JSON
`Services/AssistantRuntime.cs`	Catálogo de motores, perfiles, selección y etiquetas
`Services/AgentResilience.cs`	Reintentos, backoff, timeout, circuit breaker
`Services/AgentObservability.cs`	Trazas persistentes y control de acceso
`Services/Permissions.cs`	RBAC y niveles de lectura
`Components/Modules/AgentChat.razor`	UI del asistente, selector, fuentes y estados
`Components/Modules/Search.razor`	Pestaña Buscador + Asistente
`Models/AgentTrace.cs`	Metadatos de observabilidad (sin contenido)

Conclusión

El agente es un **asistente documental con evidencia**: recupera solo lo autorizado, responde con lenguaje natural o reglas, y siempre intenta respaldar la respuesta con una página concreta. Su valor no está en “saber mucho”, sino en **no inventar y poder demostrar de dónde sale cada afirmación**.